

ACTIVIDAD 8: ALGORITMOS VORACES (GREEDY)

Objetivo

Aplicar la estrategia de algoritmos voraces (Greedy) para resolver problemas de optimización, además de analizar el comportamiento y complejidad de los algoritmos implementados.

1. Problema: Mochila Fraccional

Estrategia Greedy Utilizada

Para maximizar el valor total sin exceder la capacidad, se utiliza una estrategia basada en la densidad de valor (valor/peso). Los pasos son:

- Calcular el ratio de valor por unidad de peso para cada objeto.
- Ordenar los objetos de manera descendente según este ratio.
- Seleccionar los objetos en orden. Si un objeto cabe completamente, se añade; de lo contrario, se toma la fracción necesaria para agotar la capacidad restante.

Análisis de Complejidad

- **Temporal:** $O(n \log n)$. El factor dominante es el ordenamiento de los n objetos. El proceso de selección posterior es lineal $O(n)$.
- **Espacial:** $O(n)$ para el almacenamiento de los datos de los objetos y sus ratios calculados.

Ejemplo de Ejecución

Entrada: Capacidad 50. Objetos A(60, 10), B(100, 20), C(120, 30).

Resultado: Objeto A completo, Objeto B completo, y 20/30 partes del Objeto C. Valor total obtenido: 240.

2. Problema: Cobertura de Antenas

Estrategia Greedy Utilizada

El objetivo es minimizar el número de antenas con rango R necesarias para cubrir todas las casas en una línea recta. La estrategia consiste en:

- Ordenar las casas por su posición de menor a mayor.
- Localizar la primera casa no cubierta (x) y colocar una antena en la posición $x + R$.
- Esto garantiza que la casa x sea cubierta por el extremo izquierdo del radio de la antena, extendiendo la cobertura lo más posible hacia la derecha (hasta $x + 2R$).

- Omitir todas las casas que caigan dentro de este nuevo rango y repetir el proceso.

Análisis de Complejidad

- **Temporal:** $O(n \log n)$ si las posiciones requieren ordenamiento inicial. La lógica de colocación es lineal $O(n)$.
- **Espacial:** $O(1)$ en memoria adicional si se opera sobre el arreglo original, o $O(n)$ si se requiere una estructura de datos para las posiciones.

Ejemplo de Ejecución

Entrada: Casas [1, 2, 7, 11, 20, 21, 30], Rango $R = 5$.

Resultado: Antenas en posiciones 6 y 25. Cantidad total de antenas: 2.

Entregables de Código

El código fuente completo implementado en Java, incluyendo los comentarios de legibilidad y los commits realizados durante el desarrollo, se encuentran disponibles en el repositorio de GitHub vinculado en la plataforma de entrega.